





考点1 树形推断

- ① $N = n_0 + n_1 + n_2 + \dots + n_n = 0 * n_0 + 1 * n_1 + 2 * n_2 + \dots + n * n_n + 1$
- ② 非空二叉树满足 $n_0 = n_2 + 1$
- ③ 度为 m 的树第 i 层上最多 m^{i-1} 个结点
- ④ 高度为 h 的满 m 叉树一共有 $\frac{m^h - 1}{m - 1}$ 个结点
- ⑤ 具有 n 个结点的完全二叉树的高度为 $\lceil \log_2(n + 1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$
- ⑥ 对完全二叉树按照层序进行编号， $i = 1, 2, \dots, n$ ：
 - ① i 为偶数时，父结点为 $i/2$ ， i 是父结点的左孩子； i 为奇数时，父结点为 $(i-1)/2$ ， i 是父结点的右孩子
 - ② i 的左孩子为 $2i$ 或不存在， i 的右孩子为 $2i+1$ 或不存在
 - ③ $i \leq \lfloor n/2 \rfloor$ 时， i 为分支结点； $i > \lfloor n/2 \rfloor$ 时， i 为叶子结点
 - ④ 若 n 为奇数，则每个分支结点都有左右孩子；若 n 为偶数，则 $i = n/2$ （最后一个分支结点）只有左孩子
- ⑦ 完全二叉树中度为 1 的结点 (n_1) 只可能有 1 个或者 0 个
- ⑧ 树越高越细（每层结点数量少），越矮越胖（每层结点数量多）
- ⑨ 完全二叉树只有最后两层可能出现叶结点



考点1 树形推断

【2010】在一棵度为4的树T中, 若有20个度为4的结点, 10个度为3的结点, 1个度为2的结点, 10个度为1的结点, 则树T的叶结点的个数是

A. 41

B. 82

C. 113

D. 122



考点1 树形推断

【2016】若森林F有15条边，25个结点，则F包含树的个数是

A. 8

B. 9

C. 10

D. 11



考点1 树形推断

【2009】已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是

A. 39

B. 52

C. 111

D. 119



考点1 树形推断

【2011】若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是

A. 257

B. 258

C. 384

D. 385



考点1 树形推断

【2018】设一棵非空完全二叉树T的所有叶结点均位于同一层，且每个非叶结点都有2个子结点。若T有k个叶结点，则T的结点总数是

A. $2k-1$

B. $2k$

C. k^2

D. 2^k-1



考点1 树形推断

【2022】若三叉树T中有244个结点（叶结点的高度为1），则T的高度至少是

A. 8

B. 7

C. 6

D. 5



考点1 树形推断

【2016】若一棵非空 k ($k \geq 2$) 叉树 T 中的每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题并给出推导过程。

- 1) 若 T 有 m 个非叶结点，则 T 中的叶结点有多少个？
- 2) 若 T 的高度为 h （单结点的树 $h=1$ ），则 T 的结点数最多为多少个？最少为多少个？



考点1 树形推断

习题训练

1. 某完全二叉树共有100个结点，则该树中叶子结点的个数最多可能是

A. 35

B. 37

C. 50

D. 63

考点1 树形推断

BOK数据结构强化



选择题考频：7/16，分值：2

大题考频：1/16，分值：10

习题训练

2. 已知一棵完全二叉树有128个叶子结点，根节点的深度为1，那么这棵树可能的最大深度是

A. 7

B. 8

C. 9

D. 10



考点1 树形推断

习题训练

3. 一棵完全二叉树顺序存储在一维数组 $B[1...n]$ 中，14号结点和30号结点的共同祖先是

A. 6

B. 7

C. 2

D. 10



考点1 树形推断

习题训练

4. 给定结点总数 n ，则不能确定叶子结点数量的是

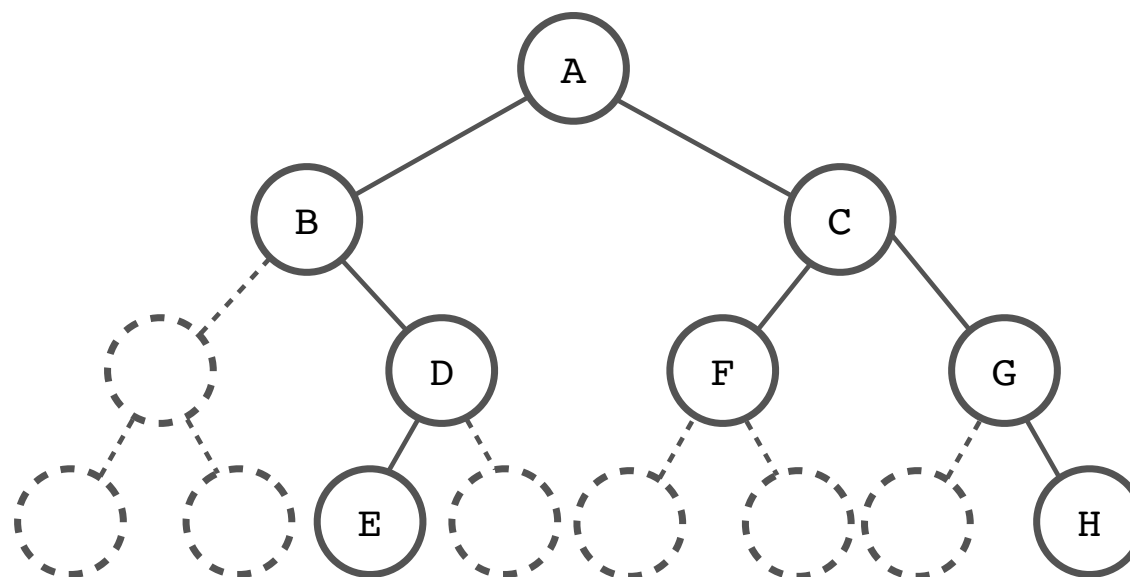
- A. 满二叉树
- B. 完全二叉树
- C. 哈夫曼树
- D. 二叉搜索树**





考点2 树的存储

①二叉树的两种存储方式



8	A	B	C	NULL	D	F	G	NULL	NULL	E	NULL	NULL	NULL	NULL	H
---	---	---	---	------	---	---	---	------	------	---	------	------	------	------	---

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

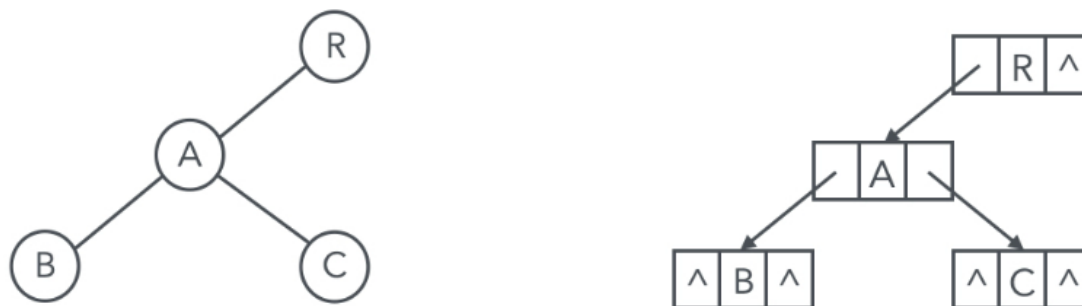
A	B	C	NULL	D	F	G	NULL	NULL	E	NULL	NULL	NULL	NULL	H	
---	---	---	------	---	---	---	------	------	---	------	------	------	------	---	--

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15



考点2 树的存储

①二叉树的两种存储方式

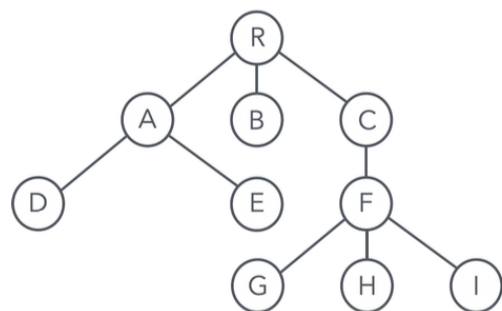


n 个结点的二叉链表有 $n+1$ 个空链域

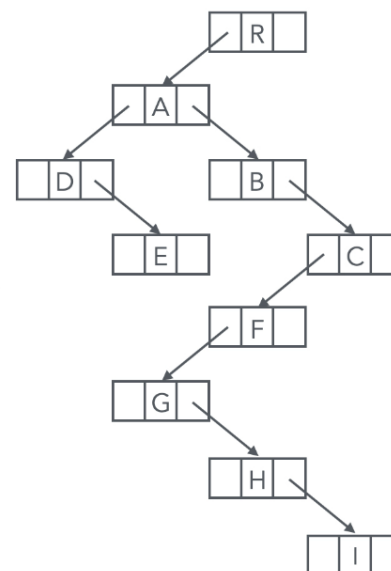


考点2 树的存储

②树的三种存储方式



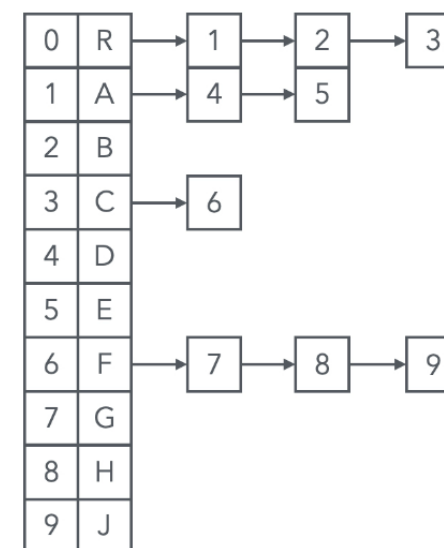
左孩子右兄弟



存储内容	-1	0	0	0	1	1	3	6	6	6
数组下标	0	1	2	3	4	5	6	7	8	9
对应节点	R	A	B	C	D	E	F	G	H	I

双亲表示法

孩子兄弟表示法



孩子表示法



考点2 树的存储

②树的三种存储方式

	双亲表示法	孩子表示法	孩子兄弟表示法
特点	以一组连续的存储单元存储树的结点的双亲	将每个结点的孩子视为以单链表存储的线性表 n个结点又组成另一个基于顺序表的线性表	对树的每个结点, 按照“左孩子右兄弟”的原则转换为二叉树 并用二叉链表存储
优点	寻找双亲方便	寻找孩子方便	寻找孩子方便
缺点	寻找孩子需要遍历整个结构	寻找双亲需要遍历整个结构	寻找双亲需要遍历整个结构



考点2 树的存储

【2020】对于任意一棵高度为5且有10个结点的二叉树，若采用顺序存储结构保存，每个结点占1个存储单元（仅存放结点的数据信息），则存放该二叉树需要的存储单元数量至少是

A. 31

B. 16

C. 15

D. 10

考点2 树的存储

BOK数据结构强化



选择题考频: 1/16, 分值: 2

大题考频: 0/16

习题训练

5. 具有 n 个结点的三叉树用三叉链表表示, 则树中空指针域的个数为

A. $3n+1$

B. $2n+1$

C. $3n-1$

D. $3n$

考点2 树的存储

BOK数据结构强化



选择题考频: 1/16, 分值: 2

大题考频: 0/16

习题训练

6. 利用二叉链表存储树, 则根结点的右指针

A. 指向最左孩子

B. 指向最右孩子

C. 为空

D. 非空

考点2 树的存储

BOK数据结构强化



选择题考频: 1/16, 分值: 2

大题考频: 0/16

习题训练

7. 有一棵使用双亲表示法存储的树，则错误的选项是

- A. 这棵树的高度是4
- B. 这棵树一共有5个叶子结点
- C. 这种存储结构需要一组连续的空间来存储结点
- D. 这棵树转换为二叉树后有7层**

0	K	-1
1	A	0
2	B	0
3	C	0
4	D	0
5	E	2
6	F	2
7	G	6





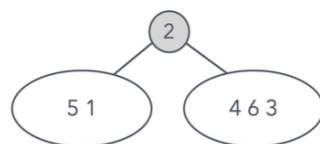
考点3 唯一确定一棵二叉树

中序 + 任何一种遍历序列 = 一棵确定的二叉树



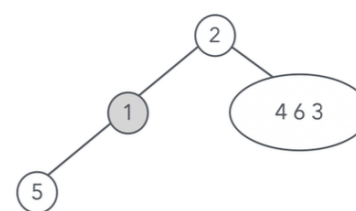
考点3 唯一确定一棵二叉树

中序 + 先序



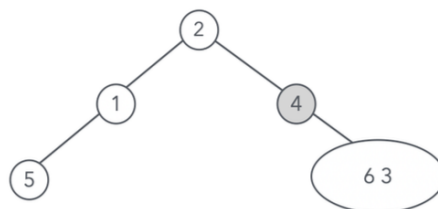
①

先序	2	1	5	4	3	6
中序	5	1	2	4	6	3



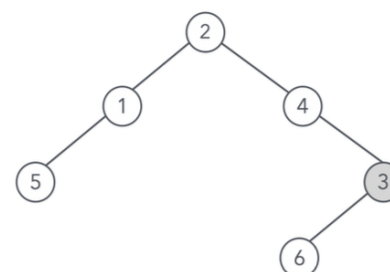
②

先序		1	5	4	3	6
中序	5	1		4	6	3



③

先序	4	3	6
中序	4	6	3



④

先序	3	6
中序	6	3



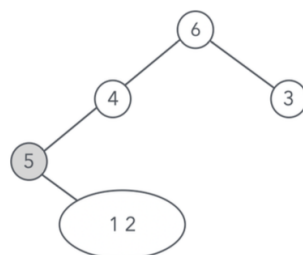
考点3 唯一确定一棵二叉树

中序 + 后序



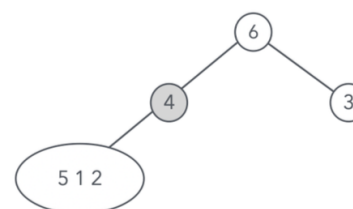
①

后序	2	1	5	4	3	6
中序	5	1	2	4	6	3



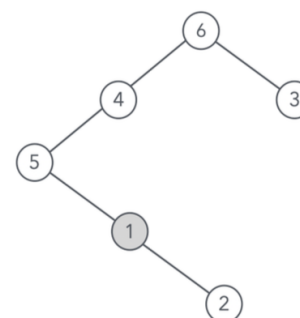
③

后序	2	1	5
中序	5	1	2



②

后序	2	1	5	4
中序	5	1	2	4



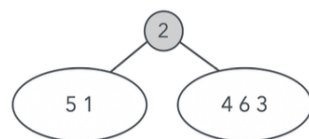
④

后序	2	1
中序	1	2



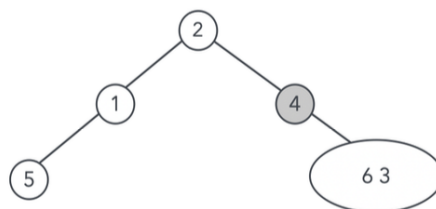
考点3 唯一确定一棵二叉树

中序 + 层序



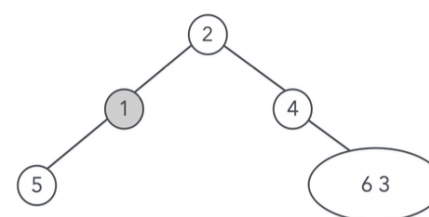
①

层序	2	1	4	5	3	6
中序	5	1	2	4	6	3



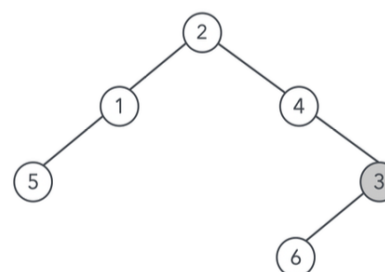
③

层序	4	3	6
中序	4	6	3



②

层序		1	4	5	3	6
中序	5	1		4	6	3



④

层序	3	6
中序	6	3



考点3 唯一确定一棵二叉树

【2011】一棵二叉树的前序遍历序列和后序遍历序列分别为1，2，3，4和4，3，2，1，该二叉树的中序遍历序列不会是

- A. 1，2，3，4
- B. 2，3，4，1
- C. 3，2，4，1**
- D. 4，3，2，1



考点3 唯一确定一棵二叉树

【2012】若一棵二叉树的前序遍历序列为a, e, b, d, c, 后序遍历序列为b, c, d, e, a, 则根结点的孩子结点

- A. 只有e**
- B. 有e、b
- C. 有e、c
- D. 无法确定

考点3 唯一确定一棵二叉树

BOK数据结构强化



选择题考频：2/16，分值：2

大题考频：0/16

习题训练

8. 已知某二叉树的先序遍历为：ABDECF，中序遍历为：DBE AFC，则该二叉树的后序遍历为

A. DEBFCA

B. EDBFCA

C. DEBCFA

D. EDBCFA





考点4 遍历和线索二叉树

①二叉树的四种遍历

```
void PreOrderTraverse(BiTree T)
{
    if(T) {
        cout << T->data;
        PreOrderTraverse(T->lchild);
        PreOrderTraverse(T->rchild);
    }
}
```

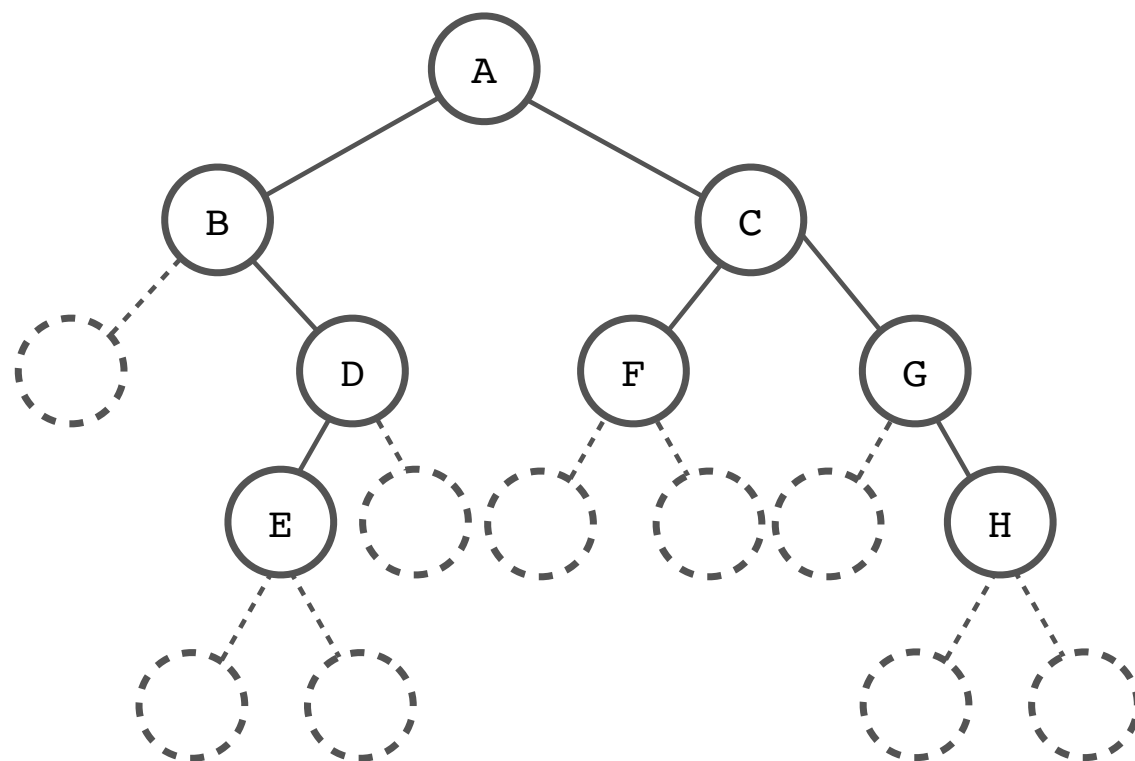
遍历过程中每个结点都会经过3次

第1次经过时访问并输出它

然后开始遍历左子树

第2次经过是因为它的左子树已遍历完毕

第3次经过是因为它的右子树已遍历完毕



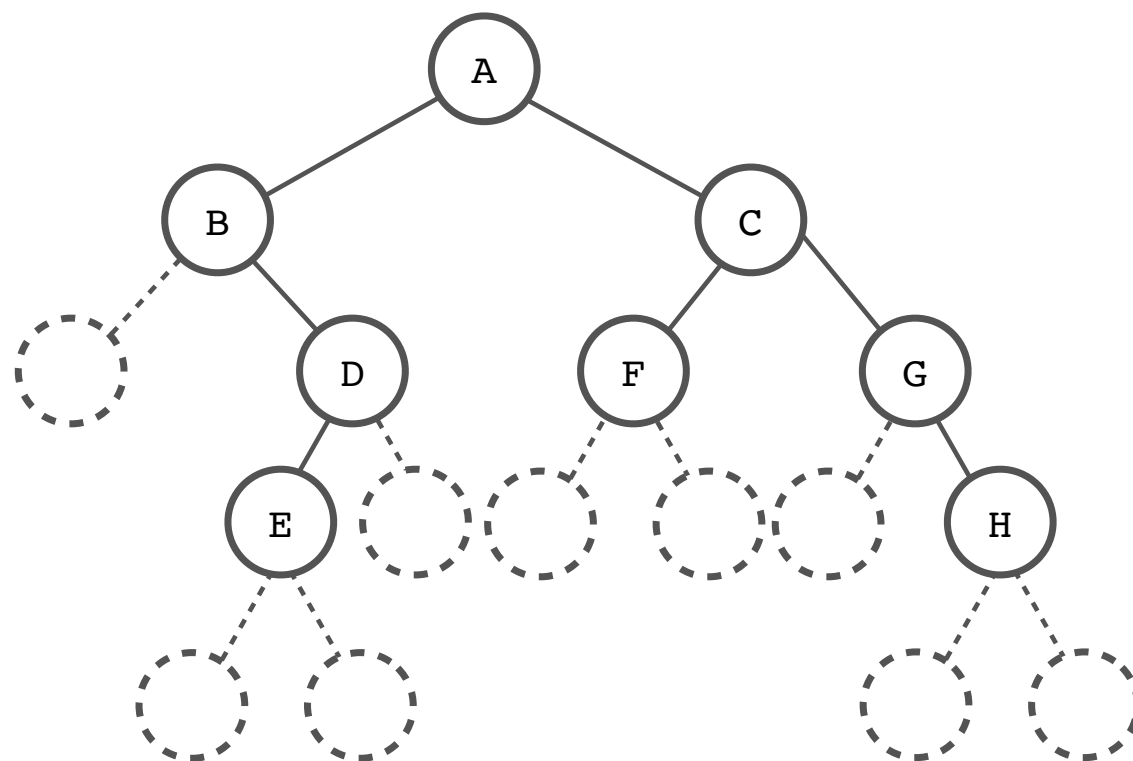


考点4 遍历和线索二叉树

①二叉树的四种遍历

```
void InOrderTraverse(BiTree T)
{
    if(T) {
        InOrderTraverse(T->lchild);
        cout << T->data;
        InOrderTraverse(T->rchild);
    }
}
```

遍历过程中每个结点都会经过3次
第1次经过是为了能访问到它的左子树
第2次经过是因为左子树已遍历完毕
此时访问并输出它
然后开始遍历右子树
第3次经过是因为它的右子树已遍历完毕



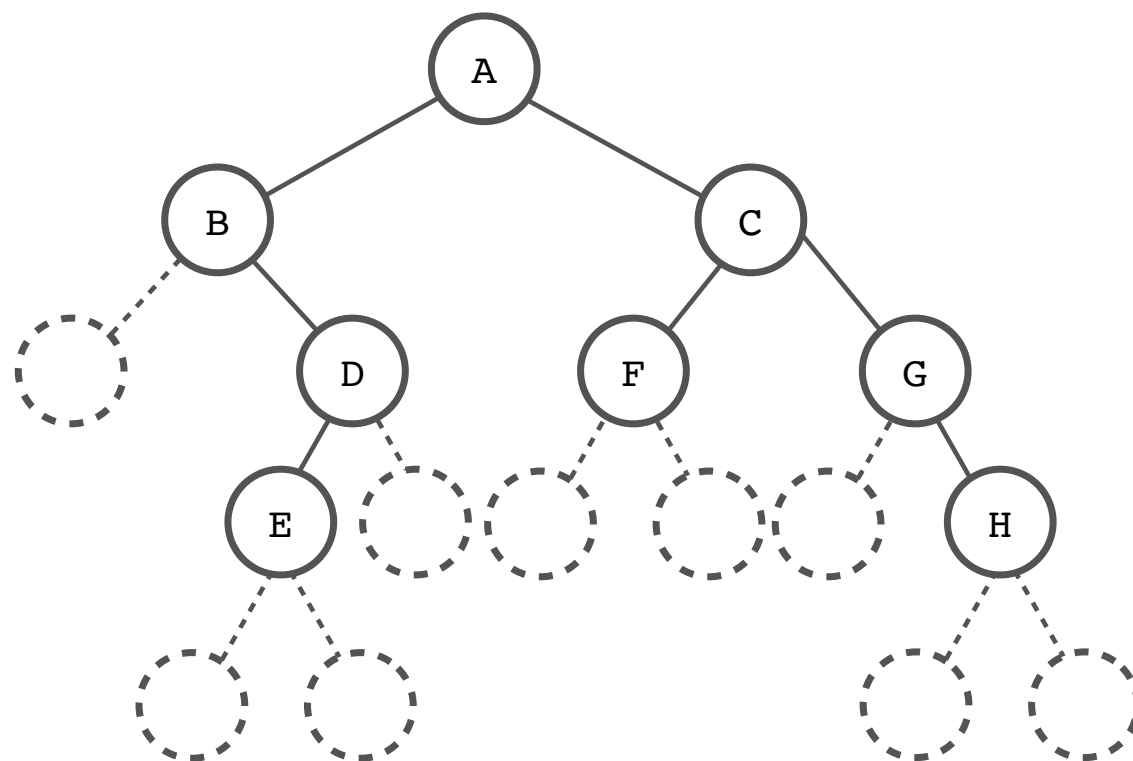


考点4 遍历和线索二叉树

①二叉树的四种遍历

```
void PostOrderTraverse(BiTree T)
{
    if(T) {
        PostOrderTraverse(T->lchild);
        PostOrderTraverse(T->rchild);
        cout << T->data; }
}
```

遍历过程中每个结点都会经过3次
第1次经过是为了能访问到它的左子树
第2次经过是为了能访问到它的右子树
第3次经过是因为左右子树已遍历完毕
此时访问并输出它





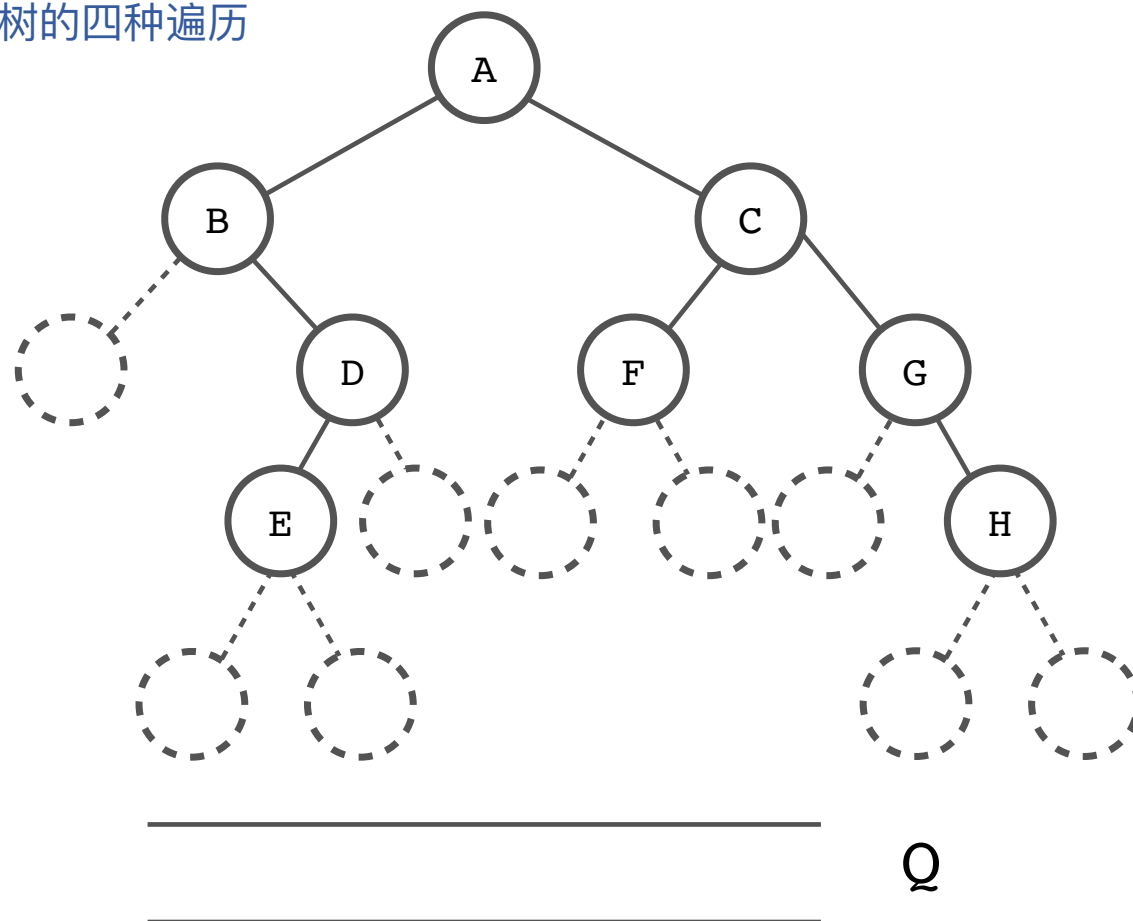
考点4 遍历和线索二叉树

①二叉树的四种遍历

```
void LevelOrder(BiTree T){  
    InitQueue(Q);  
    BiTree p;  
    EnQueue(Q,T);  
    while(!IsEmpty(Q)){  
        DeQueue(Q,p);  
        cout << p->data;  
        if(p->lchild != NULL)  
            EnQueue(Q, p->lchild);  
        if(p->rchild != NULL)  
            EnQueue(Q, p->rchild);  
    }  
}
```

实际上利用了**BFS**的思想

当队列不空时就出队队头元素并访问输出
然后将队头元素所有的邻接点（左右孩子）入队
不断重复这个过程，直到队列为空

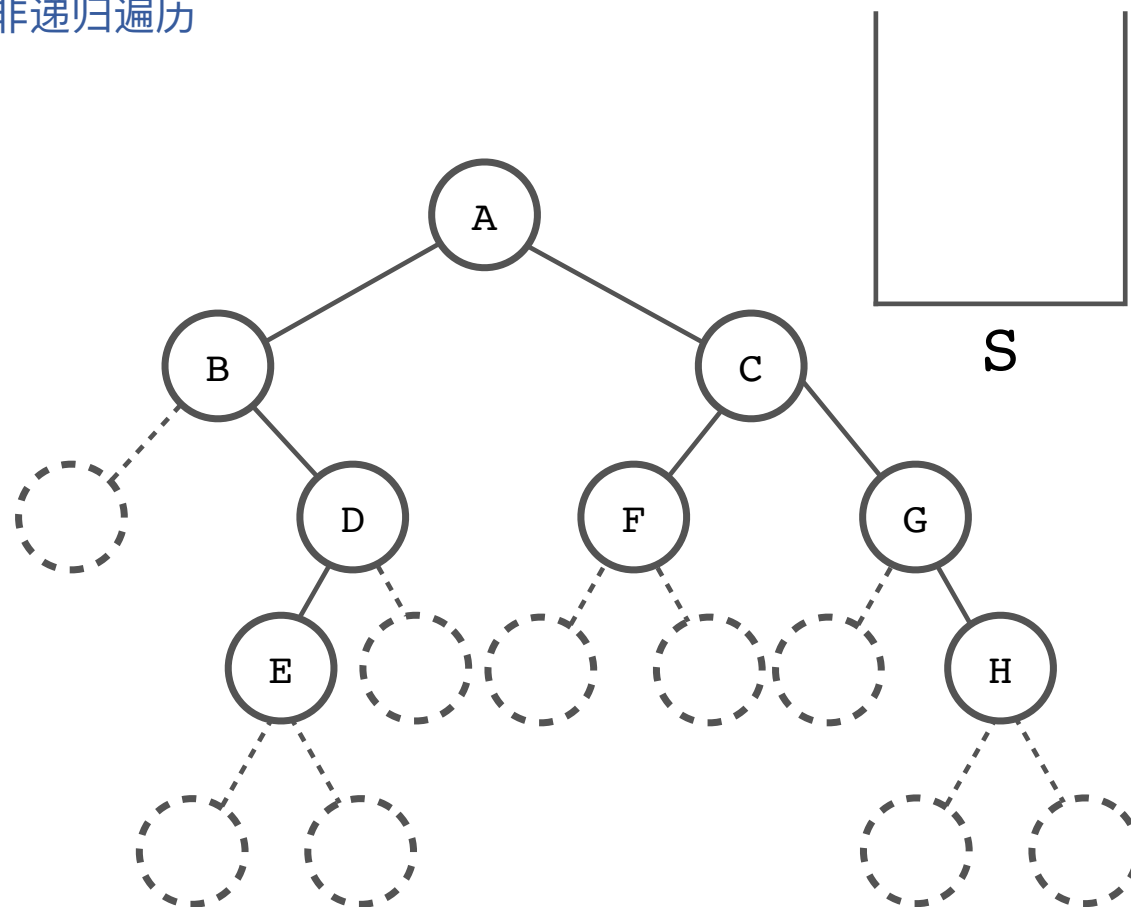




考点4 遍历和线索二叉树

②非递归遍历

```
void PreOrderTraverse(BiTree T)
{
    InitStack(S);
    BiTree p = T;
    BiTree q;
    while(p || !StackEmpty(S))
    {
        if(p)
        {
            cout << p->data;
            Push(S, p);
            p = p->lchild;
        }
        else
        {
            Pop(S, q);
            p = q->rchild;
        }
    }
}
```





考点4 遍历和线索二叉树

②非递归遍历

```
void PreOrderTraverse(BiTree T)
{
    InitStack(S);
    BiTree p = T;
    BiTree q;
    while(p || !StackEmpty(S))
    {
        if(p)
        {
            cout << p->data;
            Push(S, p);
            p = p->lchild;
        }
        else
        {
            Pop(S, q);
            p = q->rchild;
        }
    }
}
```

```
void InOrderTraverse(BiTree T)
{
    InitStack(S);
    BiTree p = T;
    BiTree q;
    while(p || !StackEmpty(S))
    {
        if(p)
        {
            Push(S, p);
            p = p->lchild;
        }
        else
        {
            Pop(S, q);
            cout << q->data;
            p = q->rchild;
        }
    }
}
```



考点4 遍历和线索二叉树

②非递归遍历

```
void PostOrderTraverse(BiTree T)
{
    InitStack(S);
    BiTree p = T, r = NULL;
    while(p || !StackEmpty(S))
    {
        if(p)
        {
            Push(S, p);
            p = p->lchild;
        }
        else
        {
            GetTop(S, q);
            if(q->rchild && q->rchild != r)
                p = q->rchild;
            else
            {
                Pop(S, q);
                cout << q->data;
                r = q;
                p = NULL;
            }
        }
    }
}
```

r手动记录“右子树是否访问过”

取栈顶元素 q

如果【 q 的右孩子非空】且【不是上一个访问输出的结点】

就说明 q 的右子树还没遍历过

此时不能输出 q ，而是先去遍历 q 的右子树

如果【 q 的右孩子为空】或【 q 的右孩子刚被访问过】

说明此时可以正常访问输出 q

并用 r 手动记录刚访问过的结点 q



考点4 遍历和线索二叉树

③线索化

lchild	lTag	data	rTag	rchild
--------	------	------	------	--------

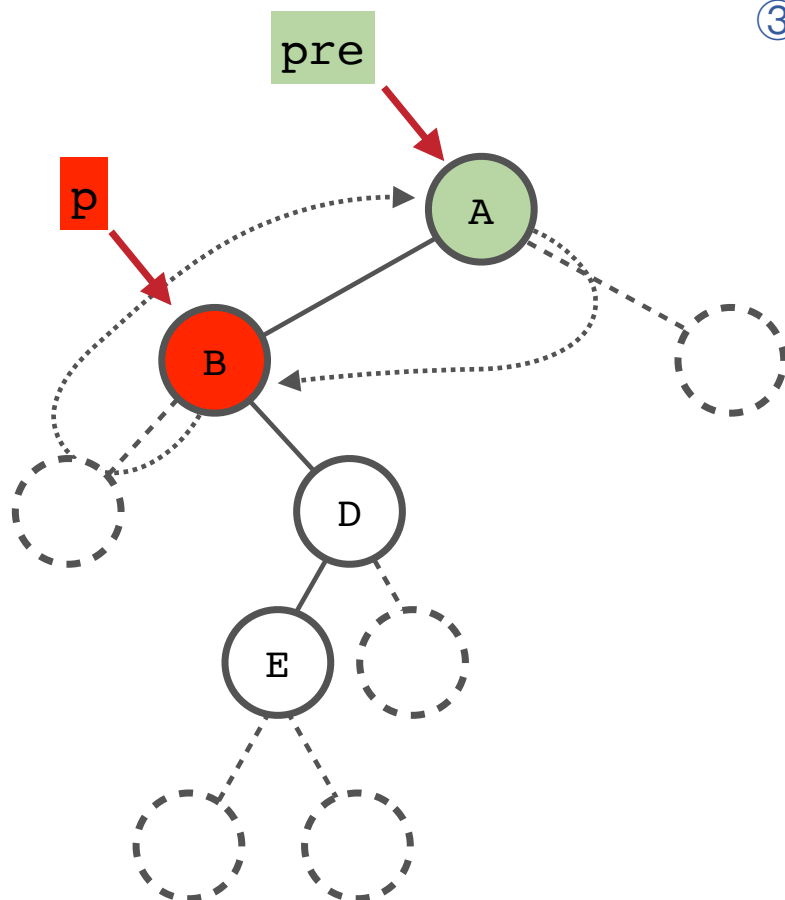
lTag=0	lchild指示结点的左孩子
lTag=1	lchild指示结点的前驱
rTag=0	rchild指示结点的右孩子
rTag=1	rchild指示结点的后继

线索二叉树是一个物理结构



考点4 遍历和线索二叉树

③线索化



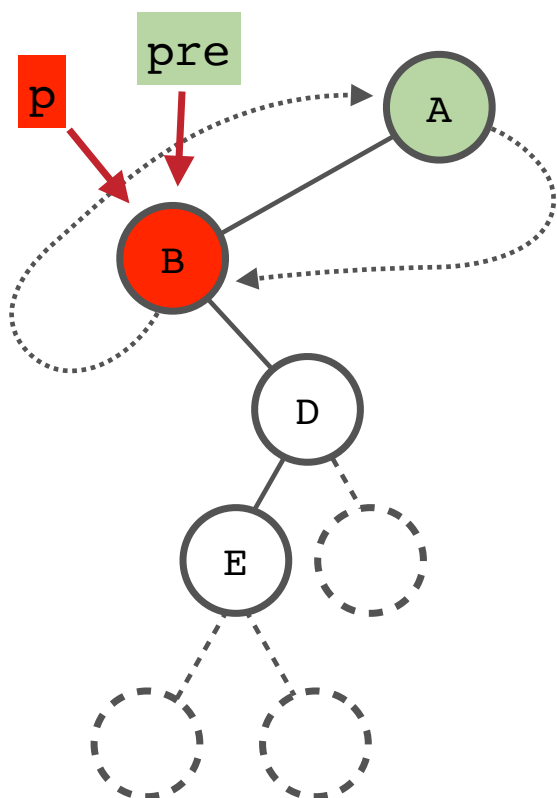
```
pre = NULL;
```

```
if(!p->lchild)
{
    p->lchild = pre;
    p->ltag = 1;
}
if(pre && !pre->rchild)
{
    pre->rchild = p;
    pre->rtag = 1;
}
pre = p;
```



考点4 遍历和线索二叉树

③线索化



```
pre = NULL;
```

```
if(!p->lchild)
{
    p->lchild = pre;
    p->ltag = 1;
}
if(pre && !pre->rchild)
{
    pre->rchild = p;
    pre->rtag = 1;
}
pre = p;
```



考点4 遍历和线索二叉树

③线索化

```
void PreOrderTraverse(BiTree T)
{
    InitStack(S);
    BiTree p = T;
    BiTree q;
    while(p || !StackEmpty(S))
    {
        if(p)
        {
            cout << p->data;
            Push(S, p);
            p = p->lchild;
        }
        else
        {
            Pop(S, q);
            p = q->rchild;
        }
    }
}
```

```
void PreOrderTraverse(BiTree T)
{
    InitStack(S);
    BiTree p = T;
    BiTree q;
    BiTree pre = NULL;
    while(p || !StackEmpty(S))
    {
        if(p)
        {
            if(!p->lchild)
            {
                p->lchild = pre;
                p->ltag = 1;
            }
            if(pre && !pre->rchild)
            {
                pre->rchild = p;
                pre->rtag = 1;
            }
            pre = p;
            cout << p->data;
            Push(S, p);
            if(p->ltag==0) p = p->lchild;
            else p = NULL;
        }
        else
        {
            Pop(S, q);
            if(q->rtag == 0)p = q->rchild;
        }
    }
}
```



考点4 遍历和线索二叉树

③线索化

```
void InOrderTraverse(BiTree T)
{
    InitStack(S);
    BiTree p = T;
    BiTree q;
    while(p || !StackEmpty(S))
    {
        if(p)
        {
            Push(S, p);
            p = p->lchild;
        }
        else
        {
            Pop(S, q);
            cout << q->data;
            p = q->rchild;
        }
    }
}
```

```
void InOrderTraverse(BiTree T)
{
    InitStack(S);
    BiTree p = T;
    BiTree q;
    BiTree pre = NULL;
    while(p || !StackEmpty(S))
    {
        if(p)
        {
            Push(S, p);
            p = p->lchild;
        }
        else
        {
            Pop(S, q);
            if(!q->lchild)
            {
                q->lchild = pre;
                q->ltag = 1;
            }
            if(pre && !pre->rchild)
            {
                pre->rchild = q;
                pre->rtag = 1;
            }
            pre = q;
            cout << q->data;
            p = q->rchild;
        }
    }
}
```

```

void PostOrderTraverse(BiTree T)
{
    InitStack(S);
    BiTree p = T, r = NULL;
    while(p || !StackEmpty(S))
    {
        if(p)
        {
            Push(S, p);
            p = p->lchild;
        }
        else
        {
            GetTop(S, q);
            if(q->rchild && q->rchild != r)
                p = q->rchild;
            else
            {
                Pop(S, q);
                cout << q->data;
                r = q;
                p = NULL;
            }
        }
    }
}

```

```

void PostOrderTraverse(BiTree T)
{
    InitStack(S);
    BiTree p = T, r = NULL;
    while(p || !StackEmpty(S))
    {
        if(p)
        {
            Push(S, p);
            p = p->lchild;
        }
        else
        {
            GetTop(S, q);
            if(q->rchild && q->rchild != r)
                p = q->rchild;
            else
            {
                Pop(S, q);
                if(!q->lchild)
                {
                    q->lchild = pre;
                    q->ltag = 1;
                }
                if(pre && !pre->rchild)
                {
                    pre->rchild = q;
                    pre->rtag = 1;
                }
                pre = q;
                cout << q->data;
                r = q;
                p = NULL;
            }
        }
    }
}

```





考点4 遍历和线索二叉树

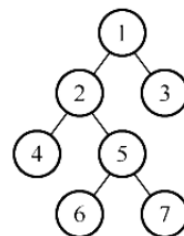
③线索化

中序线索树中，若某结点有右孩子，则它的后继结点是它右子树中最左下的结点
中序线索树中，若某结点有左孩子，则它的前驱结点是它左子树中最右下的结点



考点4 遍历和线索二叉树

【2009】给定二叉树如下图所示。设N代表二叉树的根，L代表根结点的左子树，R代表根结点的右子树。若遍历后的结点序列是3175624，则其遍历方式是

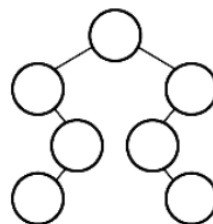


- A. LRN
- B. NRL
- C. RLN
- D. RNL**



考点4 遍历和线索二叉树

【2017】某二叉树的树形如下图所示，其后序序列为e, a, c, b, d, g, f，树中与结点a同层的结点是



A. c

B. d

C. f

D. g



考点4 遍历和线索二叉树

- 【2017】要使一棵非空二叉树的先序序列与中序序列相同，其所有非叶结点须满足的条件是
- A. 只有左子树
 - B. 只有右子树**
 - C. 结点的度均为1
 - D. 结点的度均为2



考点4 遍历和线索二叉树

【2022】若结点p与q在二叉树T的中序遍历序列中相邻, 且p在q之前, 则下列p与q的关系中, 不可能的是

- I. q是p的双亲
- II. q是p的右孩子
- III. q是p的右兄弟
- IV. q是p的双亲的双亲

A. 仅I

B. 仅III

C. 仅II、III

D. 仅II、IV



考点4 遍历和线索二叉树

【2024】若 p 、 q 和 v 均为二叉树 T 中的结点， v 有两个孩子结点， T 的中序遍历序列形如“ $\dots, p, v, q, \dots$ ”，则在下列叙述中，正确的是

A. p 没有右孩子， q 没有左孩子

B. p 没有右孩子， q 有左孩子

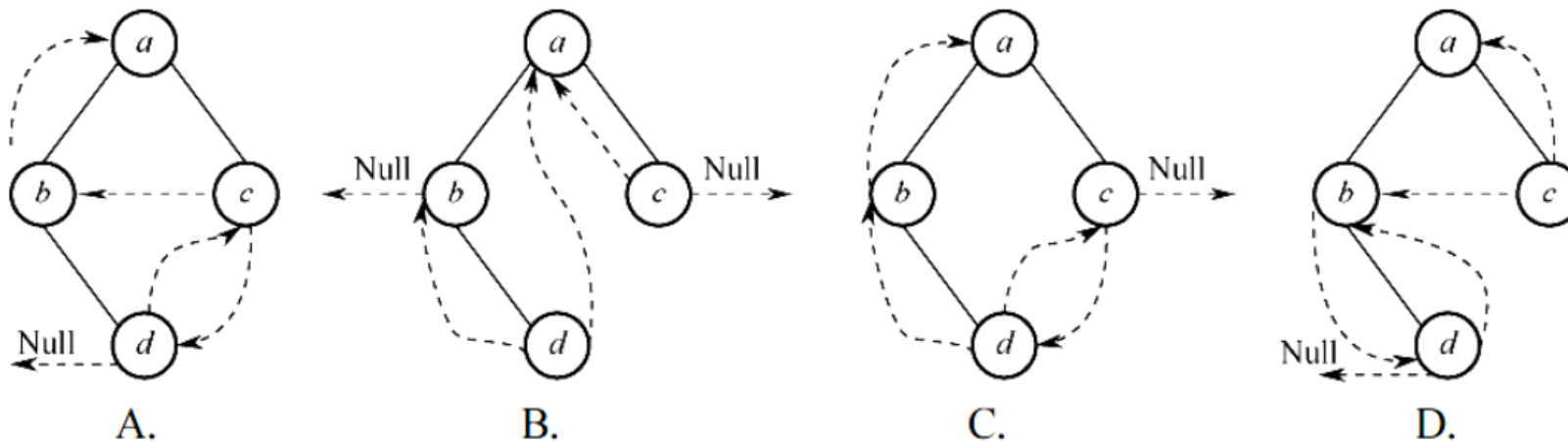
C. p 有右孩子， q 没有左孩子

D. p 有右孩子， q 有左孩子



考点4 遍历和线索二叉树

【2010】下列线索二叉树中（用虚线表示线索），符合后序线索树定义的是





考点4 遍历和线索二叉树

【2013】若x是后序线索二叉树中的叶结点，且x存在左兄弟结点y，则x的右线索指向的结点分别是

A. x的父结点

B. 以y为根的子树的最左下结点

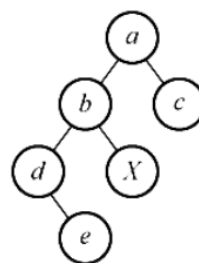
C. x的左兄弟结点y

D. 以y为根的子树的最右下结点



考点4 遍历和线索二叉树

【2014】若对下图所示的二叉树进行中序线索化，则结点x的左、右线索指向的结点分别是



A. e, c

B. e, a

C. d, c

D. b, a

考点4 遍历和线索二叉树

BOK数据结构强化



选择题考频：9/16，分值：2

大题考频：1/16，分值：15

习题训练

9. 若x是二叉中序线索树中一个有左孩子的结点，且x不为根，则x的前驱为

A. x的双亲

B. x的右子树中最左的结点

C. x的左子树中最右的结点

D. x的左子树中最右的叶结点

考点4 遍历和线索二叉树

BOK数据结构强化



选择题考频：9/16，分值：2

大题考频：1/16，分值：15

习题训练

10. 一棵中序线索二叉树共有7个结点，则该树中的线索数量是

A. 6

B. 7

C. 8

D. 13



考点4 遍历和线索二叉树

习题训练

11. 下列关于二叉树的叙述中，正确的是

- A. 若二叉树的先序遍历结果是ABCD，则其中序遍历结果可以是DABC
- B. 如果二叉树的先序遍历序列和中序遍历序列的最后一个结点相同，那么该结点一定是整棵树最右侧的叶子结点**
- C. 要交换二叉树所有分支结点的左右子树，最好利用中序遍历作为框架解决
- D. 删除一棵二叉树可以通过先序递归方式依次释放所有结点



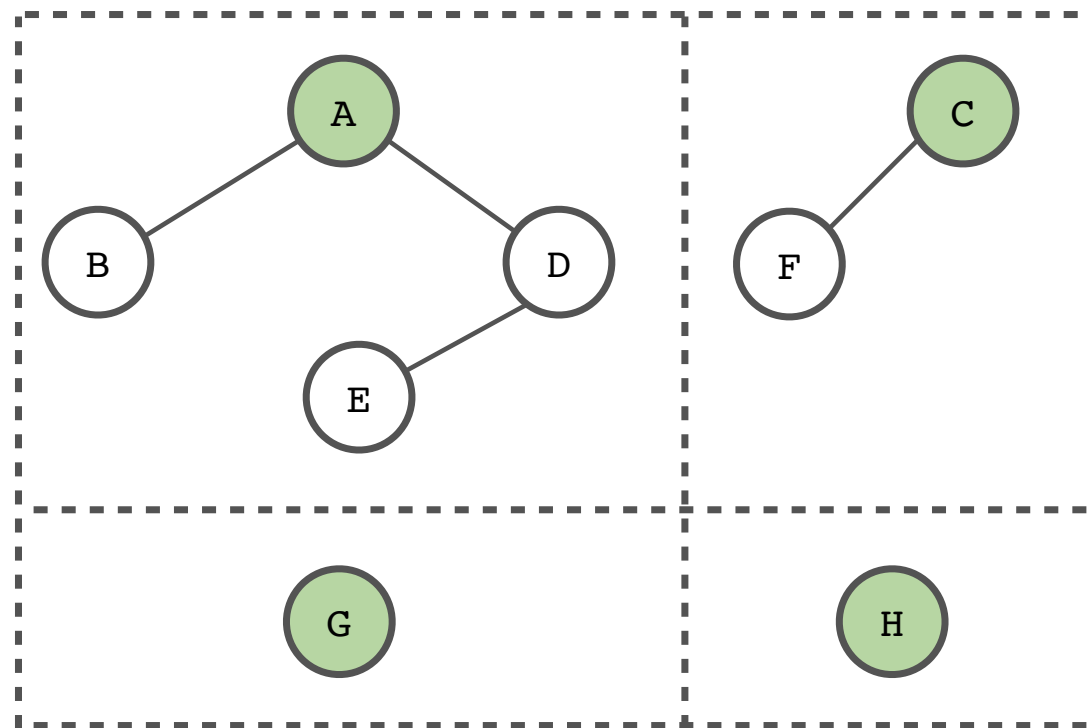
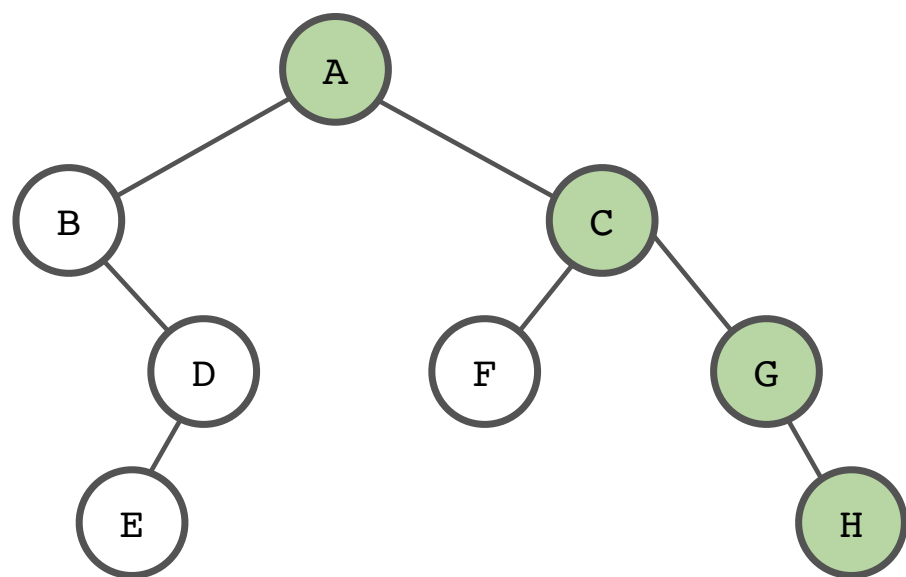


考点5 树、森林、二叉树

树和二叉树		二叉树和森林	
树 $\leftrightarrow$ 二叉树	左孩子 右兄弟	二叉树 $\rightarrow$ 森林	二叉树的根，根的右孩子，根的右孩子的右孩子...均为森林中树的根结点 其余按照左孩子右兄弟转换
		森林 $\rightarrow$ 二叉树	森林中所有树的根结点组成二叉树的根，根的右孩子，根的右孩子的右孩子... 其余按照左孩子右兄弟转换



考点5 树、森林、二叉树

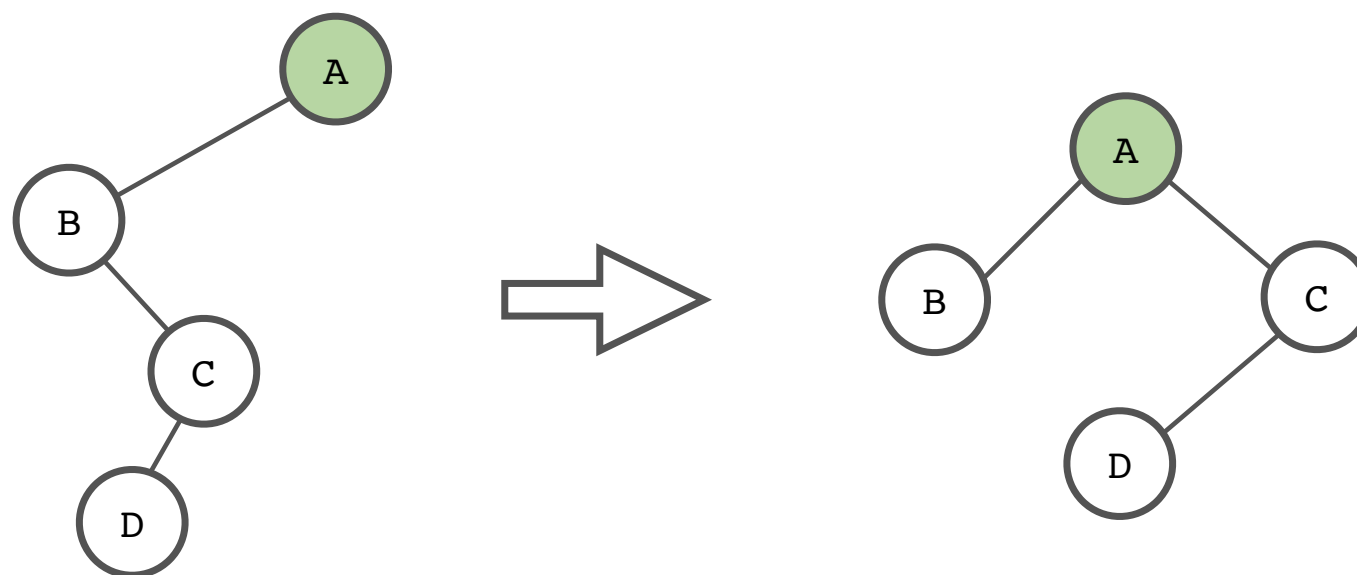


二叉树中从根结点开始，一直沿右孩子指针向下所经过的结点个数 = 对应森林中树的数量

森林中树的数量 = 对应二叉树中从根结点开始，一直沿右孩子指针向下所经过的结点个数



考点5 树、森林、二叉树

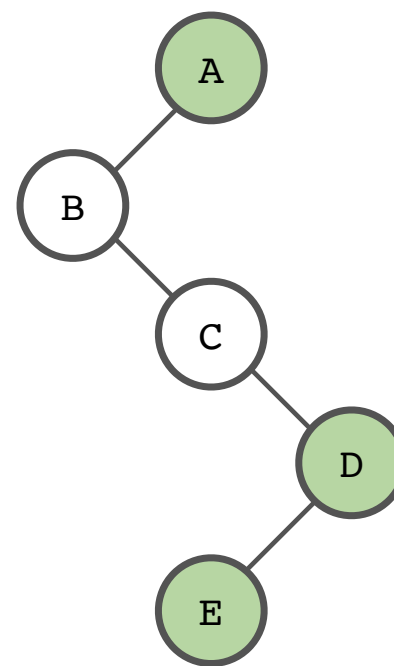
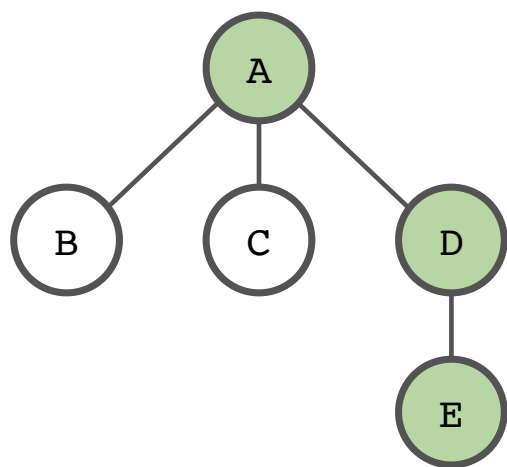


二叉树的根结点没有右子树 = 转换后是一棵树

二叉树的根结点有右子树 = 转换后是一个至少有两棵树的森林



考点5 树、森林、二叉树

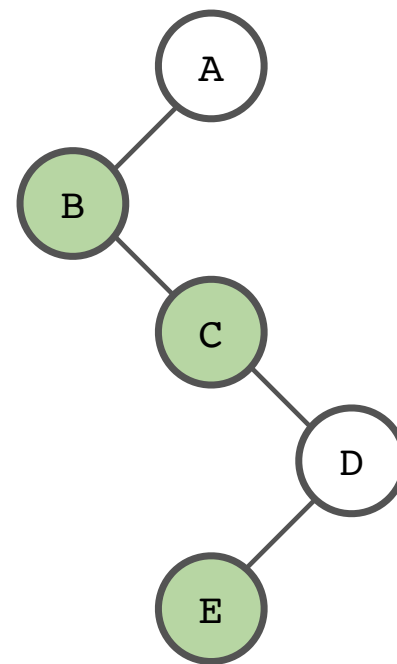
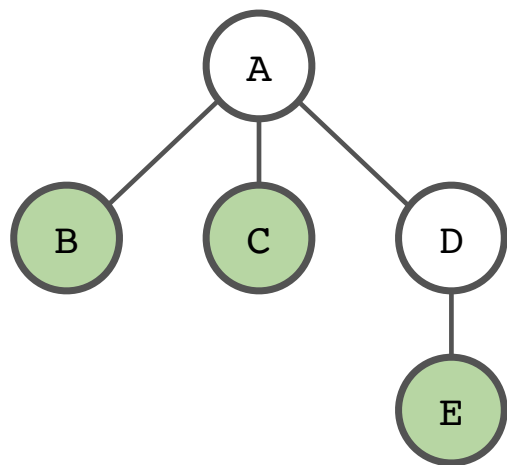


树的根结点以及每个非终端结点的最后一个孩子 = 二叉树中没有右孩子的结点

二叉树中没有右孩子的结点 = 树的根结点以及每个非终端结点的最后一个孩子



考点5 树、森林、二叉树



树的叶结点 = 二叉树中没有左孩子的结点

二叉树中没有左孩子的结点 = 树的叶结点



考点5 树、森林、二叉树

树	二叉树	森林
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历	中（后）序遍历

如果不知道如何对树和森林进行遍历，可先转化为二叉树，再对二叉树进行相应的遍历



考点5 树、森林、二叉树

【2009】将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 的父结点的父结点，则在原来的森林中， u 和 v 可能具有的关系是

- I. 父子关系
- II. 兄弟关系
- III. u 的父结点和 v 的父结点是兄弟关系

- A. 只有II
- B. I和II**
- C. I和III
- D. I、II和III



考点5 树、森林、二叉树

【2011】已知一棵有2011个结点的树，其叶结点个数为116，该树对应的二叉树中无右孩子的结点个数为

- A. 115
- B. 116
- C. 1895
- D. 1896**



考点5 树、森林、二叉树

- 【2014】将森林F转换为对应的二叉树T，F中叶结点的个数等于
- A. T中叶结点的个数
 - B. T中度为1的结点个数
 - C. T中左孩子指针为空的结点个数**
 - D. T中右孩子指针为空的结点个数



考点5 树、森林、二叉树

【2019】若将一棵树T转化为对应的二叉树BT，则下列对BT的遍历中，其遍历序列与T的后根遍历序列相同的是

- A. 先序遍历
- B. 中序遍历**
- C. 后序遍历
- D. 按层遍历



考点5 树、森林、二叉树

【2020】已知森林F及与之对应的二叉树T，若F的先根遍历序列是a, b, c, d, e, f, 后根遍历序列是b, a, d, f, e, c, 则T的后序遍历序列是

- A. b, a, d, f, e, c
- B. b, d, f, e, c, a
- C. b, f, e, d, c, a**
- D. f, e, d, c, b, a

【2021】某森林F对应的二叉树为T，若T的先序遍历序列是a, b, d, c, e, g, f, 中序遍历序列是b, d, a, e, g, c, f, 则F中树的棵数是

- A. 1
- B. 2
- C. 3**
- D. 4

考点5 树、森林、二叉树

BOK数据结构强化



选择题考频：6/16，分值：2

大题考频：0/16

习题训练

12. 设森林F对应的二叉树为BT，结点总数是m，若BT的右子树共有n个结点，则森林F中第一棵树的结点总数是

A. $m-n$

B. $m-n-1$

C. $n+1$

D. $n-1$

考点5 树、森林、二叉树

BOK数据结构强化



选择题考频: 6/16, 分值: 2

大题考频: 0/16

习题训练

13. 森林F中有4棵树, 结点个数分别为 M_1 , M_2 , M_3 , M_4 , 当F转换成二叉树之后, 该二叉树的根节点的右子树的结点总数是

A. M_1

B. $M_2 + M_3$

C. M_4

D. $M_2 + M_3 + M_4$

考点5 树、森林、二叉树

BOK数据结构强化



选择题考频: 6/16, 分值: 2

大题考频: 0/16

习题训练

14. 若森林F有7个结点, 3棵树, 则森林中的边数是

A. 3

B. 4

C. 5

D. 6

考点5 树、森林、二叉树

BOK数据结构强化



选择题考频：6/16，分值：2

大题考频：0/16

习题训练

15. 森林F对应的二叉树是B，若F有n个非终端结点，则B中右指针域为空的结点个数为

A. $n-1$

B. n

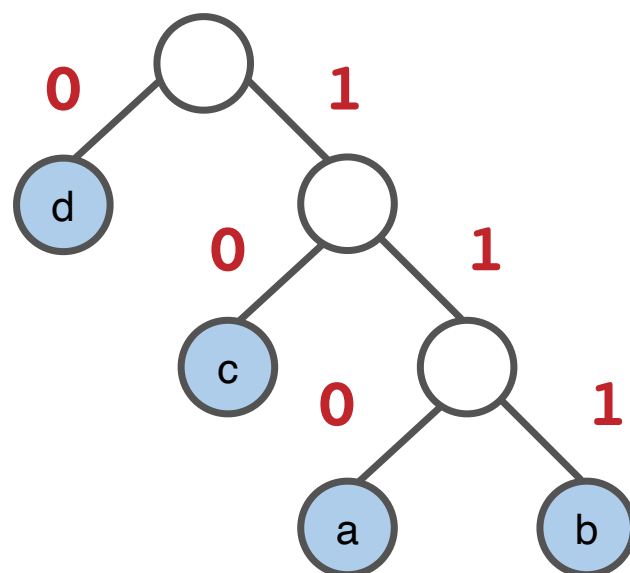
C. $n+1$

D. $n+2$





考点6 哈夫曼树和哈夫曼编码



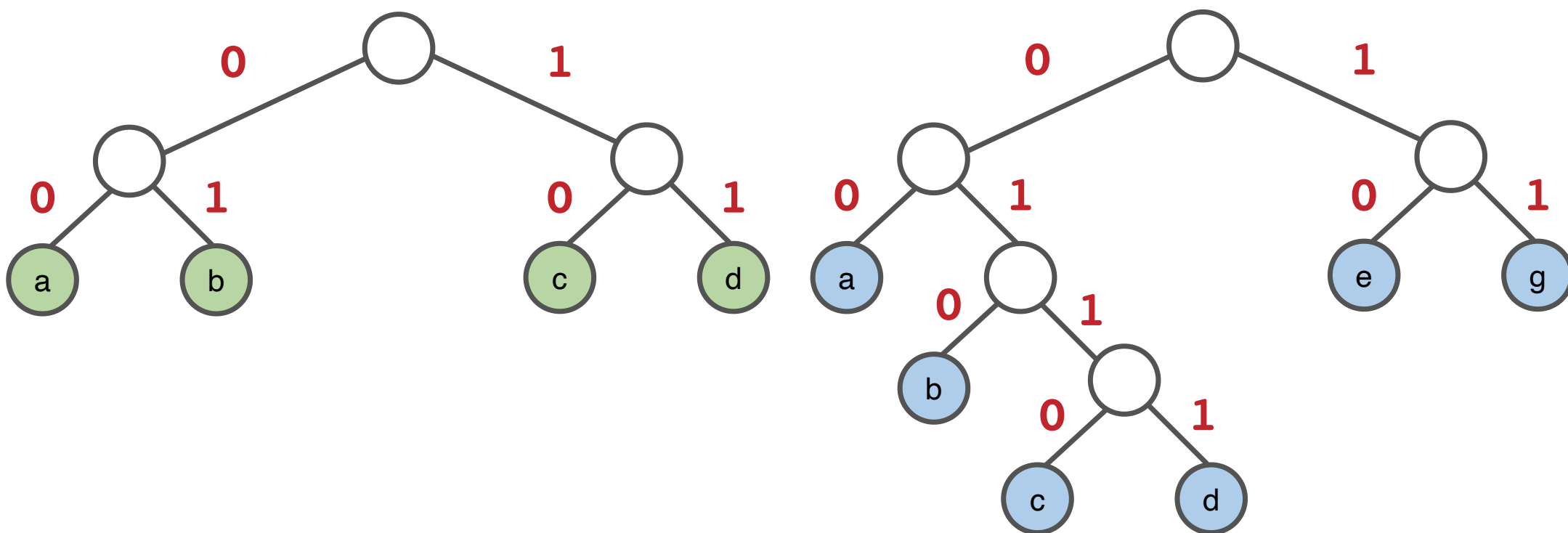
字符出现的频率

a	3
b	5
c	11
d	23

- ①将n个字符出现的频率作为权构造一棵哈夫曼树，由哈夫曼树求得的编码就是哈夫曼编码
- ②哈夫曼树的构造过程决定了字符一定是叶子结点
- ③哈夫曼编码是前缀编码，不允许某个字符的编码是另一个的前缀，
所以两个字符不能出现在树中从根到某个叶子结点的同一路径上



考点6 哈夫曼树和哈夫曼编码



定长哈夫曼编码：所有字符都位于哈夫曼树的同一层，哈夫曼编码长度相同
变长哈夫曼编码：符都位于哈夫曼树的不同层，哈夫曼编码长度不同



考点6 哈夫曼树和哈夫曼编码

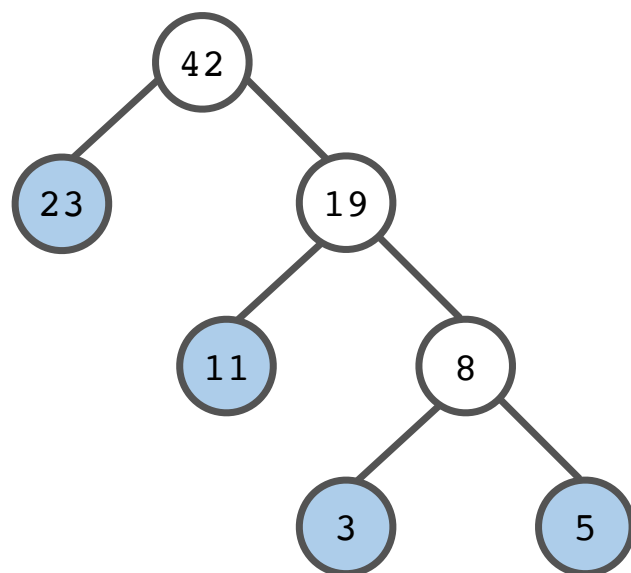
- ①若有 n 个结点，则在哈夫曼树的构造过程中新建了 $n-1$ 个结点，最终一共 $2n-1$ 个结点
- ②哈夫曼树只有度为0和2的结点，不存在度为1的结点，即 $n=n_0+n_2$
- ③非空二叉树满足 $n_0=n_2+1$ ，故哈夫曼树 $n=2n_0-1=2n_2+1$
- ④权值（频度）越小的结点一般离根结点越远，利用这条性质保证出现最多的字符拥有最短的编码
- ⑤同一组权值构造出的哈夫曼树可能不唯一

⑥ m 叉哈夫曼树：

- ①定义：只有度为0和 m 的结点的哈夫曼树
- ②构造过程：初始结点有 n_0 个，首先计算 $k = (n_0 - 1) \bmod (m - 1)$ ，如果 $k \neq 0$ ，则需要补充 $m - 1 - k$ 个权为0的结点
- ③仿照2叉哈夫曼树的构造过程，每次结合 m 个权值最小的结点，重复该过程



考点6 哈夫曼树和哈夫曼编码



$$\text{算法1: WPL} = 23 * 1 + 11 * 2 + 3 * 3 + 5 * 3 = 69$$

$$\text{算法2: WPL} = 42 + 19 + 8 = 69$$

结点的带权路径长度 = 从该结点到根之间的路径长度与结点上权值的乘积
树的带权路径长度WPL = 树中所有**叶子结点**的带权路径长度之和



考点6 哈夫曼树和哈夫曼编码

【2010】 n ($n \geq 2$) 个权值均不同的字符构成哈夫曼树，关于该树的叙述中，错误的是

A. 该树一定是一棵完全二叉树

B. 树中一定没有度为1的结点

C. 树中两个权值最小的结点一定是兄弟结点

D. 树中任意一个非叶结点的权值一定不小于下一层任意一个结点的权值



考点6 哈夫曼树和哈夫曼编码

【2015】下列选项给出的是从根分别到达两个叶结点路径上的权值序列，能属于同一棵哈夫曼树的是

- A. 24, 10, 5和24, 10, 7
- B. 24, 10, 5和24, 12, 7
- C. 24, 10, 10和24, 14, 11
- D. 24, 10, 5和24, 14, 6**



考点6 哈夫曼树和哈夫曼编码

【2019】对 n 个互不相同的符号进行哈夫曼编码。若生成的哈夫曼树共有115个结点，则 n 的值是

A. 56

B. 57

C. 58

D. 60



考点6 哈夫曼树和哈夫曼编码

【2021】若某二叉树有5个叶结点，其权值分别为10，12，16，21，30，则其最小的带权路径长度（WPL）是

A. 89

B. 200

C. 208

D. 289



考点6 哈夫曼树和哈夫曼编码

【2018】已知字符集 $\{a, b, c, d, e, f\}$ ，若各字符出现的次数分别为6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是

A. 00, 1011, 01, 1010, 11, 100

B. 00, 100, 110, 00, 0010, 01

C. 10, 1011, 11, 0011, 00, 010

D. 0011, 10, 11, 0010, 01, 000



考点6 哈夫曼树和哈夫曼编码

【2022】对任意给定的含 n ($n > 2$) 个字符的有限集 S ，用二叉树表示 S 的哈夫曼编码集和定长编码集，分别得到二叉树 T_1 和 T_2 。下列叙述中，正确的是

- A. T_1 和 T_2 的结点数相同
- B. T_1 的高度大于 T_2 的高度
- C. 出现频次不同的字符在 T_1 中处于不同的层
- D. 出现频次不同的字符在 T_2 中处于相同的层**



考点6 哈夫曼树和哈夫曼编码

【2023】在由6个字符组成的字符集s中，各字符出现的频次分别为3，4，5，6，8，10，为s构造的哈夫曼编码的加权平均长度为

A. 2.4

B. 2.5

C. 2.67

D. 2.75



考点6 哈夫曼树和哈夫曼编码

【2012】设有6个有序表A, B, C, D, E, F, 分别含有10, 35, 40, 50, 60和200个数据元素, 各表中的元素按升序排列。要求通过5次两两合并, 将6个表最终合并为1个升序表, 并使最坏情况下比较的总次数达到最小。请回答下列问题:

- 1) 给出完整的合并过程, 并求出最坏情况下比较的总次数。
- 2) 根据你的合并过程, 描述 n ($n \geq 2$) 个不等长升序表的合并策略, 并说明理由。



考点6 哈夫曼树和哈夫曼编码

【2020】若任意一个字符的编码都不是其他字符编码的前缀，则称这种编码具有前缀特性。现有某字符集（字符个数 ≥ 2 ）的不等长编码，每个字符的编码均为二进制的0、1序列，最长为 L 位，且具有前缀特性。请回答以下问题：

- 1) 哪种数据结构适宜保存上述具有前缀特性的不等长编码？
- 2) 基于你所设计的数据结构，简述从0/1串到字符串的译码过程。
- 3) 简述判定某字符集的不等长编码是否具有前缀特性的过程。

考点6 哈夫曼树和哈夫曼编码

BOK数据结构强化



选择题考频：10/16，分值：2

大题考频：2/16，分值：10

习题训练

16. 已知某哈夫曼树有7个叶子结点，则该哈夫曼树中的结点总数是

A. 13

B. 14

C. 11

D. 9

考点6 哈夫曼树和哈夫曼编码

BOK数据结构强化



选择题考频：10/16，分值：2

大题考频：2/16，分值：10

习题训练

17. 某通信协议中使用如下规则为消息分配编码，每个字符对应一组二进制串。现在已知某些字符的编码如下：A:00,B:010,C:011,D:1，则下列说法中正确的是

- A. 可以将A的编码从00改成0，而不会影响前缀码的性质
- B. 需要将D的编码从1改成10，以避免与其他字符冲突
- C. 该编码方式无法构成前缀码
- D. 该编码方式是前缀码，且无法再缩短任何编码**

考点6 哈夫曼树和哈夫曼编码

BOK数据结构强化



选择题考频：10/16，分值：2

大题考频：2/16，分值：10

习题训练

18. 下列不可能是哈夫曼树结点总数的是

A. 762

B. 761

C. 403

D. 127

考点6 哈夫曼树和哈夫曼编码

BOK数据结构强化



选择题考频：10/16，分值：2

大题考频：2/16，分值：10

习题训练

19. 对字符进行哈夫曼编码时，不可能的方案是

A. 00, 01, 10, 11

B. 00, 010, 011, 100, 101, 11

C. 00, 100, 101, 110, 111

D. 0000, 0001, 001, 01, 1

考点6 哈夫曼树和哈夫曼编码

BOK数据结构强化



选择题考频：10/16，分值：2

大题考频：2/16，分值：10

习题训练

20. 如果要求哈夫曼编码的长度小于等于4，并且现在已经对2个字符编码为0和10，则最多还可以对多少个字符进行编码？

A. 3

B. 4

C. 5

D. 6